💻 Pruebas del Backend (Node.js + Sequelize)

Aquí nos enfocamos en que nuestra API funcione correctamente y que la interacción con la base de datos sea impecable.

Herramienta	Rol en el Proyecto	¿Qué Prueba?
Jest	**El Corredor de Pruebas**	Ejecuta todas las pruebas, proporciona las funciones it() y expect(). Es el motor.
SQLite	**La Base de Datos de Mentira**	Simula una base de datos real (PostgreSQL/MySQL) de forma muy rápida y desechable, solo para la prueba. Así no "ensuciamos" la base de datos de desarrollo/producción.
Supertest	**El Cliente HTTP Simulado**	Nos permite simular llamadas HTTP (GET, POST, etc.) a nuestras rutas de Express/API. Prueba si la **ruta** devuelve el código de estado y la data correcta.

Flujo Típico de una Prueba de Integración:

1. Usamos **Jest** para ejecutar la prueba.
2. La prueba se conecta a la base de datos **SQLite**.
3. Usamos **Supertest** para enviar un POST a /usuarios.
4. Verificamos con **Jest** que el código de respuesta sea 201 y que el usuario se haya guardado correctamente en **SQLite** a través de Sequelize.

⚛️ Pruebas del Frontend (React)

Aquí la meta es asegurar que nuestros componentes se vean y se comporten como el usuario espera, sin preocuparnos de los detalles internos de React.

Herramienta	Rol en el Proyecto	¿Qué Prueba?
Jest	**El Corredor de Pruebas**	Mismo motor que el backend, pero configurado para entender JSX y React.
React Testing Library (RTL)	**El Usuario Virtual**	**La más importante.** Nos permite interactuar con el componente *tal como lo haría un usuario*. Ej: hacer clic en un botón con el texto "Guardar", verificar si aparece un mensaje de error. ¡No probamos el estado interno, probamos lo que ve el usuario!
Mock Service Worker (MSW)	**El Camarero que Falsifica la Carta**	Intercepta las llamadas de red (API). En lugar de llamar al backend real, **MSW** devuelve una respuesta falsa que definimos. Esto nos permite probar si el componente

Herramienta	Rol en el Proyecto	¿Qué Prueba?
		maneja bien la carga de datos, errores de red o datos vacíos, sin que el backend tenga que estar encendido.

Flujo Típico de una Prueba de Componente:

1. Usamos **MSW** para decirle: "cuando este componente llame a /productos, responde con esta lista de 3 productos".
2. Usamos **RTL** para renderizar el componente.
3. Usamos **RTL** para verificar si el usuario ve en la pantalla los 3 productos que **MSW** simuló.

📚 Estructura de Documentación de Pruebas

Una buena práctica es organizar las pruebas de la misma manera que está organizado el código que prueban.

Nivel	Descripción	Ejemplo de Carpeta/Archivo
Raíz	Carpeta principal de todas las pruebas.	/test o /src/__tests__
Por Capa/Dominio	Dividir por si es Front, Back, o por la parte del negocio.	/test/backend/usuarios, /test/frontend/components
Archivo	Archivo de prueba para un módulo de código específico.	user.service.test.js, LoginForm.test.jsx

Dentro del Archivo de Prueba:

Usa describe() para agrupar las pruebas y it() o test() para la prueba individual.

JavaScript

```
// user.service.test.js (Backend)


describe('UserService', () => {
  // 1. Configuración (conexión a DB, mocks, etc.)
  beforeAll(() => { /* setup */ });

  // 2. Grupo de Pruebas para una Función/Ruta
  describe('GET /api/users', () => {
    // 3. Prueba Individual
```

```
    test('Debería retornar todos los usuarios (código 200)', async () => {
      // ... código de la prueba con Supertest y expect
    });

    test('Debería retornar un error 404 si no hay usuarios', async () => {
      // ... código de la prueba
    });
  });
});
```